Analyse du budget dans le jeu de Hex

JABER Aya MOHAMMAD Khalil ALHAZZAA Laith
HABART Pierre

L3 Informatique
Chargé de TP : Alexis Lechervy(Groupe 1B)

27 janvier 2026

UNIVERSITÉ
CAEN
NORMANDIE

Plan

1. Introduction
2. Architecture logicielle
3. État d'Avancement
4. Organisation et Planning
5. Perspectives et conclusion
6. Annexe

Présentation du projet

- Basé sur le jeu de **Hex**
- **plateau hexagonal** de taille N x N → relier les deux bords
  1. Bleu : Est et Ouest
  2. Rouge : Nord et Sud
- Chaque *tour* consiste à **placer un pion sur une case libre**
- Avantage au premier joueur (Bleu)
- Impossible d'avoir un match nul → toujours **un gagnant**

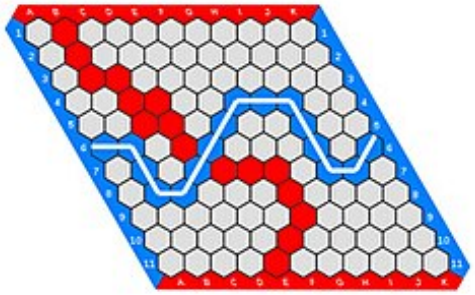

Figure – Victoire de bleu

Problématiques

Question Scientifique

Quelle influence **le rapport de budget** (temps de calcul/itérations) entre les deux joueurs a-t-il sur leur **probabilité de victoire** en fonction de la **taille de la grille** ?

Sous-questions :

- Peut-on compenser l'avantage du premier joueur en augmentant le budget du second ?
- À quel ratio de budget cet avantage est-il annulé ?
- Ce seuil dépend-il de la taille de la grille ?

Budget = Nombre de simulations MCTS allouées par coup

Approche : Algorithme MCTS

MCTS (Monte carlo Tree Search) :

- Explore l'arbre de jeu via simulations aléatoire
- Se concentre progressivement sur les coups prometteurs
- Plus le budget est élevé, meilleure est la décision

4 phases de l'algorithme :

1. **Sélection** : Choix itératif des noeuds les plus prometteurs.
2. **Expansion** : Ajouter un noeud non exploré
3. **Simulation** : Jouer aléatoirement jusqu'à la fin
4. **Rétropropagation** : Mettre à jour les statistiques

Optimisation RAVE :

Extension utilisant les statistiques AMAF(All Moves As First) pour améliorer la convergence vers les bons coups

Architecture logicielle

Organisation modulaire en 6 package :

- **model** : plateau, parties, joueurs, Stratégies d'IA (Random, MCTS, RAVE)
- **algo** (Algorithmes d'IA) : MCTS, RAVE (Optimisation)
- **utils** : détection de victoire (Union-Find, DFS)
- **view** : affichage plateau, visualisations
- **analyse** : Statistiques, génération courbes
- **tests** : tests unitaires, et d'intégration

Diagramme de classes UML

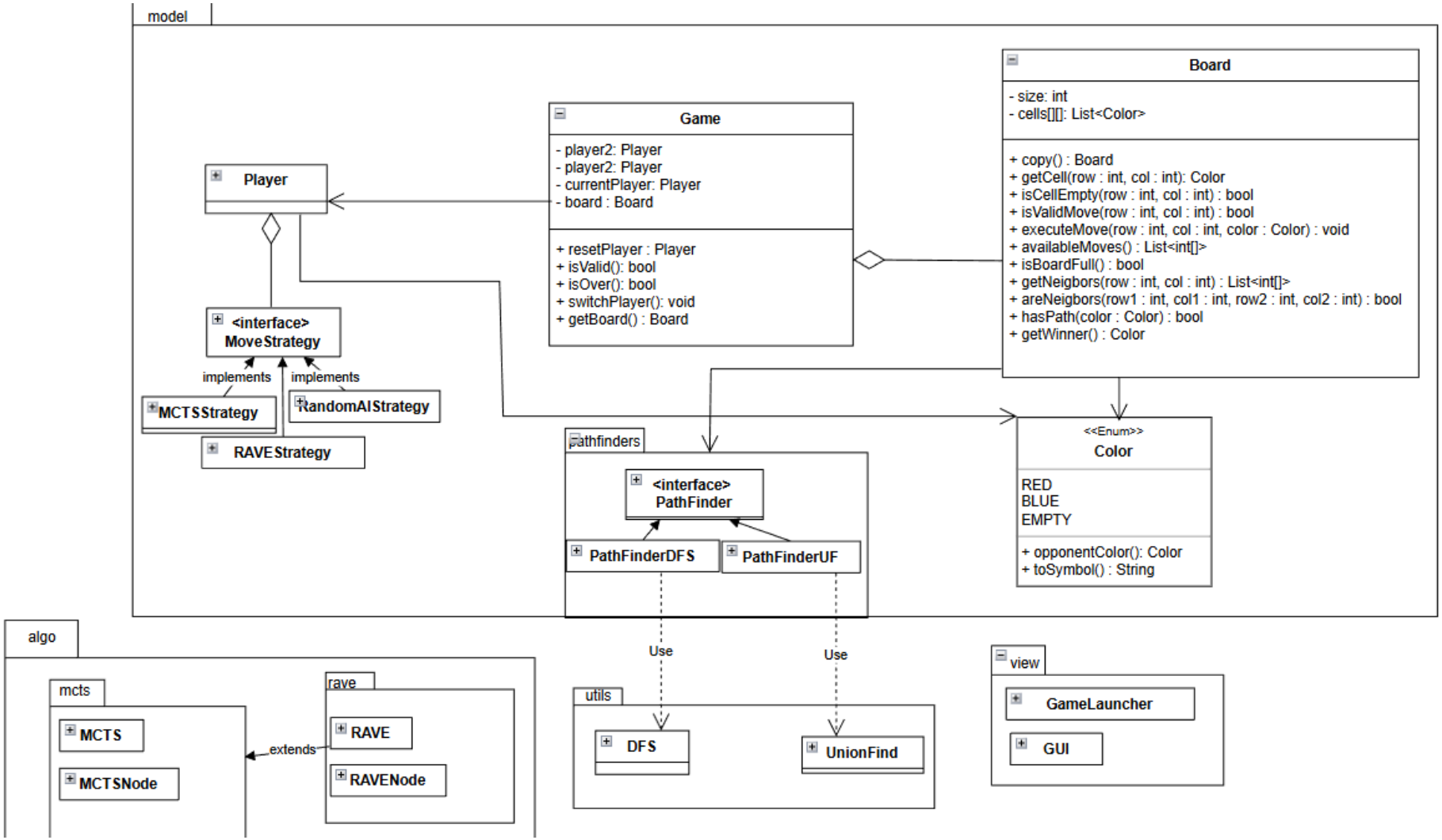

Figure – UML

État d'avancement – Réalisé et en cours

Modélisation du jeu

- ✓ Grille hexagonale (taille variable)
- ✓ Gestion des coups et validation des règles
- ✓ Voisins hexagonaux (6 directions)
- ✓ Structure de donnée (Union-Find)
- ✓ Affichage du plateau

IA de base

- ✓ Joueurs Aléatoire fonctionnel.
- ... **Intégration :** Liaison du Union-Find avec le plateau (En cours).

Infrastructure

- ✓ Architecture modulaire

État d'avancement – à faire

Détection de victoire (Début février : 1 semaine)

- Implémentation DFS > alternative

Algorithme d'IA (Mi-février : 3 à 4 semaines)

- Implémentation MCTS
- Tests MCTS vs Random
- Implémentation RAVE
- Tests comparatifs MCTS vs RAVE

Expérimentations (Début à mi-mars : 1 à 2 semaines)

- Tests avec budgets variés
- Tailles de grilles multiples
- Comparaison des rapports de budget

Analyse et rapport (Mi à fin mars : 1 semaine)

- Analyse statistique des résultats
- Visualisations des performances (courbes, heatmaps)

Répartition des Tâches

Modélisation du jeu Travail collectif :

- Board (Aya et Laith), Game (Aya et Pierre), Player (Khalil), Color (Khalil), RandomAIStrategy(Khalil)
- Spécialisation : Union-Find (Pierre)
- DFS (Aya)

Algorithmes d'IA Binôme :

- Binôme 1 MCTS : (noeuds + algorithme)
- Binôme 2 RAVE :(noeuds + algorithme)

Tests et expérimentations Travail collectif :

- Tests unitaires : chacun sur ses composants
- Expérimentations et analyses

Perspectives et conclusion

Conclusion :

- Base solide posée
- Prochaines étapes claires (MCTS, RAVE, expérimentations)
- **Objectif** : Répondre scientifiquement à l'influence du budget

Perspectives (si temps disponible) :

- Adapter le code à d'autres jeux (Reverse Hex, Quarto)
- Optimisations avancées de MCTS (Heuristique, transposition table, parallélisation)
- Interface graphique améliorée avec animations

Merci pour votre attention !

Questions ?

Annexe : Zoom Technique Union-Find

Problème : Parcourir le graphe (BFS/DFS) à chaque simulation est trop lent. **Solution :** Structure Union-Find avec compression de chemin.

Optimisation Spécifique Hex

Utilisation de **Noeuds Virtuels** (Top, Bottom, Left, Right) pour vérifier la victoire en $O(1)$.

Schéma des connexions aux bords virtuels